

GameService Architecture

Chess Project - Board & GameService Communication Design Document

SD2 Project Group 15 | April 2026

1. Architecture Overview

The architecture follows a **delegation pattern** where Board handles only UI rendering and input detection, while GameService owns all game logic, state, and board mutations. Communication is synchronous via method calls that return a MoveResult value object.

```

Game (entry point)
|
|--- creates ---> Board (UI layer)
|--- creates ---> GameService (logic layer)
|
|   Board -----uses-----> GameService
|   |
|   |   handleSquareClick()   |   handleAction() -> MoveResult
|   |   refreshBoard()       |   getLegalMoves()
|   |   drawPiece()          |   isCorrectTurn()
|   |
|   |                           Piece.getPossibleMoves()
|   |                           (raw moves, unfiltered)
|   |
|   |
|   MoveResult (immutable return value - no shared mutable state)
  
```

Key principle: Board never mutates `piecesOnTheBoard[][]` for game moves. Every move goes through `GameService.handleAction()`, which internally updates the board array, game state, and returns a `MoveResult` describing what happened.

2. Class Diagram

● New method/field
 ● Moved from Board
 ● Kept (unchanged)
 ● Removed

Board

UI Layer - Rendering & Input Only

FIELDS

```

- boardPositions : IndexPosition[][] {static} - piecesOnTheBoard :
Piece[][] - squares : JPanel[][] selected : Piece // REMOVED moved to
GameService + gameService : GameService // NEW - reference to logic layer

```

METHODS - KEPT (UI RESPONSIBILITY)

```

+ setupBoard(window: JFrame) : void + setUpMatrix() : void +
setUpPiecesOnTheBoard() : void + initialDrawOfPieces() : void +
detectMouseClickedPosition(mouseEvent: MouseEvent) : void +
drawPiece(piece: Piece) : void + refreshBoard() : void +
getPixelsToDraw(piece: Piece) : IndexPosition {static} +
getPiecesOnTheBoard() : Piece[][] + getPieceAt(row: int, col: int) :
Piece + setPieceAt(row: int, col: int, piece: Piece) : void

```

METHODS - MODIFIED

```

+ handleSquareClick(nextMove: IndexPosition) : void // SIMPLIFIED: now
delegates entirely to GameService // Calls gameService.handleAction(),
reads MoveResult, updates UI

```

METHODS - REMOVED (MOVED TO GAMESERVICE)

```

isMovePossible(possibleMoves, nextMove) : boolean // -> GameService -
moveSelectedPiece(nextMove: IndexPosition) : void // -> GameService -
capturePiece(nextMove: IndexPosition) : void // -> GameService -
selectPiece(row: int, col: int) : void // -> GameService -
getSelected() : Piece // -> GameService

```

GameService

Logic Layer - Game Rules, State & Board Mutation

FIELDS - GAME STATE

```

- selected : Piece // from Board - currentTurn : ColorForChessPieces =
WHITE - lastMoveFrom : IndexPosition // for en passant - lastMoveTo :
IndexPosition // for en passant - lastMovedPiece : Piece // for en
passant - whiteKingMoved : boolean = false // castling rights -
blackKingMoved : boolean = false - whiteRookKingsideMoved : boolean =
false // rook at (0,7) - whiteRookQueensideMoved : boolean = false //
rook at (0,0) - blackRookKingsideMoved : boolean = false // rook at (7,7)
- blackRookQueensideMoved : boolean = false // rook at (7,0) -

```

```
positionHistory : Map<String, Integer> // for threefold repetition -
halfMoveClock : int = 0 // for 50-move rule
```

METHODS - CORE (ENTRY POINT)

```
+ handleAction(clicked: IndexPosition, board: Piece[][]) : MoveResult //
THE single entry point. Board calls ONLY this method for game actions. //
Internally handles: selection, validation, move execution, // state
updates, check/checkmate/draw detection.
```

METHODS - MOVED FROM BOARD

```
- isMovePossible(possibleMoves: IndexPosition[], target: IndexPosition) :
boolean - executeMove(piece: Piece, target: IndexPosition, board: Piece[]
[]) : void // renamed from moveSelectedPiece - mutates the board array +
getSelected() : Piece
```

METHODS - NEW: LEGAL MOVE FILTERING

```
+ getLegalMoves(piece: Piece, board: Piece[][]) : IndexPosition[] // Gets
raw moves via piece.getPossibleMoves(board), // then FILTERS OUT moves
that would leave own king in check. // Also appends castling moves (for
King) and en passant moves (for Pawn). // This single method handles pin
logic implicitly: // a pinned piece's moves that expose the king are
filtered out. - filterMovesForCheck(piece: Piece, rawMoves:
IndexPosition[], board: Piece[][]) : IndexPosition[] // For each
candidate move, simulates it and checks if own king // is still in check.
Removes illegal moves. This is the core // mechanism that enforces pin
rules automatically.
```

METHODS - NEW: CHECK & ATTACK DETECTION

```
+ isInCheck(color: ColorForChessPieces, board: Piece[][]) : boolean //
Finds the king of given color, checks if any enemy piece // attacks that
square. + isSquareAttacked(square: IndexPosition, byColor:
ColorForChessPieces, board: Piece[][]) : boolean // Checks if ANY piece
of 'byColor' can reach the given square. // Used by: isInCheck, castling
validation, king move filtering. + findKing(color: ColorForChessPieces,
board: Piece[][]) : IndexPosition // Scans board for king of given color.
Returns its position.
```

METHODS - NEW: GAME END DETECTION

```
+ isCheckmate(color: ColorForChessPieces, board: Piece[][]) : boolean //
Returns true if 'color' is in check AND has no legal moves // for ANY of
their pieces. Iterates all pieces of that color, // calls getLegalMoves()
for each - if all return empty, it's checkmate. + isStalemate(color:
ColorForChessPieces, board: Piece[][]) : boolean // Returns true if
'color' is NOT in check but has no legal moves. + isDraw(board: Piece[]
[]) : boolean // Checks all draw conditions: // - Stalemate (no legal
```

```

moves, not in check) // - Insufficient material // - Threefold repetition
(same position 3 times) // - 50-move rule (50 moves without pawn move or
capture) - hasNoLegalMoves(color: ColorForChessPieces, board: Piece[][])
: boolean // Helper: returns true if no piece of given color has any
legal move. - hasInsufficientMaterial(board: Piece[][]) : boolean // K vs
K, K+B vs K, K+N vs K, K+B vs K+B (same color bishops)

```

METHODS - NEW: SPECIAL MOVES

```

+ canCastleKingside(color: ColorForChessPieces, board: Piece[][]) :
boolean // Checks: king not moved, rook not moved, squares between empty,
// king not in check, king doesn't pass through attacked square. +
canCastleQueenside(color: ColorForChessPieces, board: Piece[][]) :
boolean // Same as kingside but for queenside rook. - executeCastle(king:
Piece, target: IndexPosition, board: Piece[][]) : void // Moves both king
and rook. Called from executeMove when castling detected. +
isEnPassantMove(pawn: Piece, target: IndexPosition) : boolean // Checks
if the move is an en passant capture by comparing // target with the en
passant target square. + getEnPassantTarget() : IndexPosition // Returns
the en passant target square (the square the capturing // pawn moves TO),
or null if en passant is not available. // Calculated from lastMoveFrom,
lastMoveTo, lastMovedPiece. - executeEnPassant(pawn: Piece, target:
IndexPosition, board: Piece[][]) : IndexPosition // Moves capturing pawn,
removes captured pawn from its actual square. // Returns the position of
the captured pawn (for UI update).

```

METHODS - NEW: STATE MANAGEMENT

```

- switchTurn() : void // Toggles currentTurn between WHITE and BLACK. +
getCurrentTurn() : ColorForChessPieces + isCorrectTurn(piece: Piece) :
boolean // Returns true if piece color matches currentTurn. -
updateCastlingRights(movedPiece: Piece, from: IndexPosition) : void //
Called after each move. Sets hasMoved flags for kings/rooks. -
updateMoveHistory(piece: Piece, isCapture: boolean) : void // Updates
halfMoveClock and positionHistory for draw detection. -
simulateMove(piece: Piece, target: IndexPosition, board: Piece[][]) :
Piece[][] // Creates a COPY of the board, applies the move on the copy,
// and returns it. Used by filterMovesForCheck to test if a move // would
leave the king in check without mutating the real board. -
boardToString(board: Piece[][]) : String // Serializes board state to a
string for threefold repetition tracking.

```

MoveResult

Immutable Value Object - Returned by GameService

FIELDS

```

+ moved : boolean // true if a move was actually made + isChecked : boolean
// opponent's king is now in check + isCheckedmate : boolean // opponent is
checkmated - game over + isDraw : boolean // game is a draw + drawReason
: String // "stalemate", "insufficient material", // "threefold
repetition", "50-move rule" + isCastle : boolean // move was a castling
move + rookFrom : IndexPosition // rook's original position (castling) +
rookTo : IndexPosition // rook's new position (castling) + isEnPassant :
boolean // move was en passant capture + capturedPawnPos : IndexPosition
// position of captured pawn (en passant) + isPromotion : boolean // pawn
reached last rank + promotionPosition : IndexPosition // where promotion
happens

```

METHODS

```

+ static noop() : MoveResult // returns a result with moved=false // Used
when a click results in no move (selection, reselection, invalid) +
getters for all fields

```

Game

Entry Point - Modified

FIELDS

```

- window : JFrame - board : Board - gameService : GameService // NEW -
created here, passed to Board

```

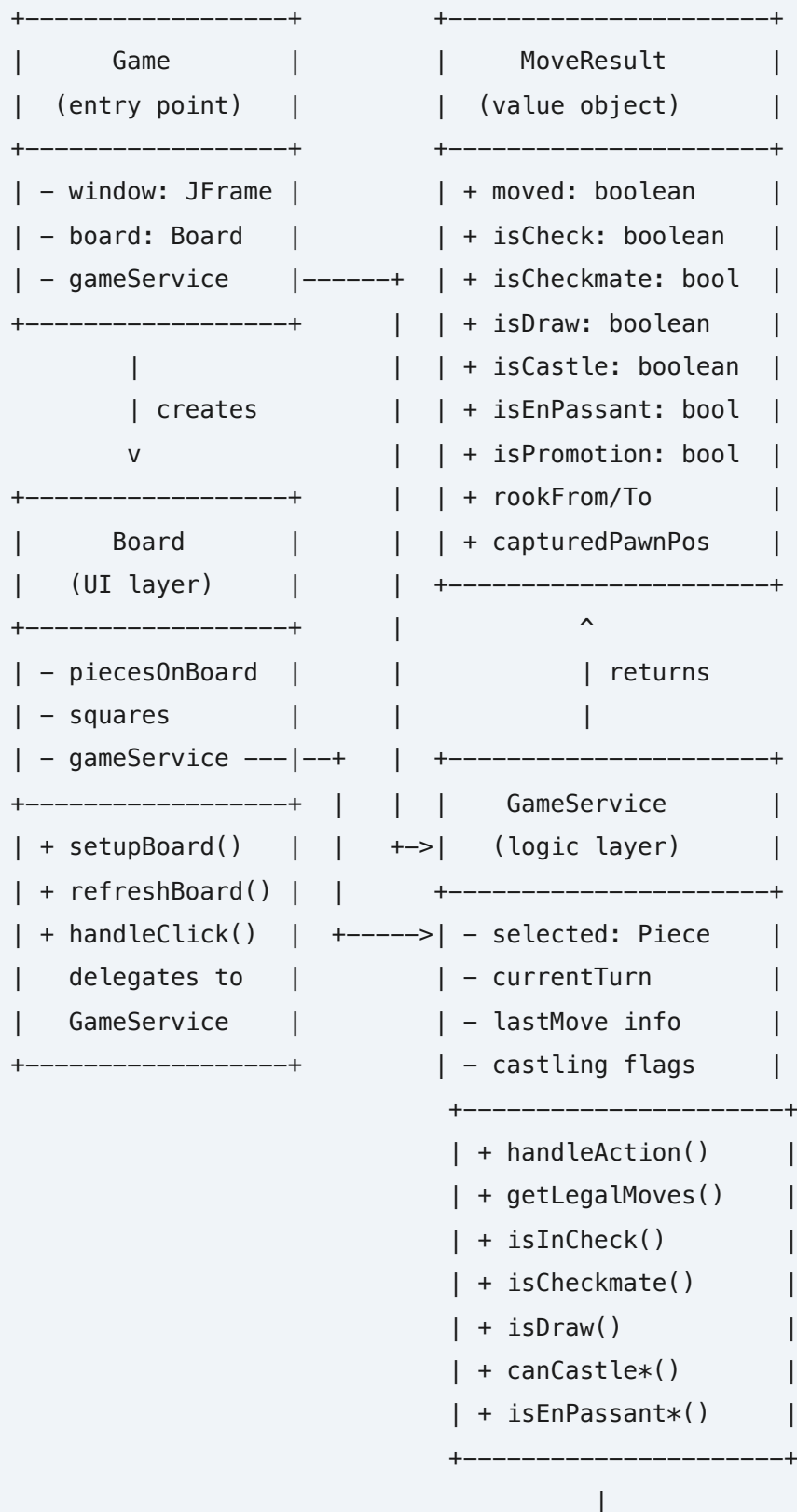
CONSTRUCTOR CHANGE

```

// Game creates both Board and GameService, then connects them: //
gameService = new GameService(); // board = new Board(gameService); //
Board receives GameService via constructor

```

3. Relationships Diagram



```

| calls
v
+-----+
| Piece (abstract) |
+-----+
| + getPossibleMoves() |
| (raw, unfiltered) |
+-----+
| King | Queen | Rook |
| Bishop | Knight | Pawn |
+-----+

```

4. Communication Flows

4.1 Normal Move

- 1 User clicks a square. `Board.detectMouseClickedPosition()` translates pixels to `IndexPosition`.
- 2 `Board.handleSquareClick()` calls `gameService.handleAction(clicked, piecesOnTheBoard)`.
- 3 **GameService** internally:
 - a) Checks if it's a selection, reselection, or move attempt.
 - b) Calls `getLegalMoves(selected, board)` which internally calls `piece.getPossibleMoves(board)` then filters via `filterMovesForCheck()`.
 - c) If valid, calls `executeMove()` to mutate the board array.
 - d) Calls `switchTurn()`, `updateCastlingRights()`, `updateMoveHistory()`.
 - e) Checks `isInCheck()`, `isCheckmate()`, `isDraw()` for the opponent.
 - f) Builds and returns `MoveResult`.
- 4 `Board` reads the returned `MoveResult`:
 - Calls `refreshBoard()` to redraw pieces.
 - If `result.isCheckmate` → shows game over dialog.
 - If `result.isDraw` → shows draw dialog.
 - If `result.isCastle` → rook already moved in array, refresh handles it.
 - If `result.isPromotion` → shows promotion picker dialog.

4.2 Castling Move

- 1 User selects King, clicks target 2 squares away (e.g. King at e1 clicks g1).
- 2 `Board` calls `gameService.handleAction(g1, board)`.
- 3 `GameService.getLegalMoves(king, board)` already includes g1 as a legal move (because `canCastleKingside()` returned true). `executeMove()` detects it's a castle and calls `executeCastle()` which moves **both** King (e1→g1) and Rook (h1→f1) in the board array.
- 4 Returns `MoveResult` with `isCastle=true, rookFrom=(0,7), rookTo=(0,5)`. `Board` calls `refreshBoard()` and both pieces appear in new positions.

4.3 En Passant

- 1 Opponent's pawn moved 2 squares forward on previous turn. `GameService` stored this in `lastMoveFrom`, `lastMoveTo`, `lastMovedPiece`.
- 2 User selects own pawn adjacent to opponent's pawn. `getLegalMoves()` checks `getEnPassantTarget()` and adds the en passant capture square to legal moves.
- 3 User clicks the en passant square. `executeMove()` detects en passant, calls `executeEnPassant()`: moves capturing pawn diagonally, removes captured pawn from its actual square.
- 4 Returns `MoveResult` with `isEnPassant=true, capturedPawnPos=(row, col)`.

4.4 Check Leading to Checkmate

- 1 White moves a piece. `GameService.executeMove()` completes the move.
- 2 `isInCheck(BLACK, board)` returns **true**.
- 3 `hasNoLegalMoves(BLACK, board)` iterates ALL black pieces, calls `getLegalMoves()` for each. Every move is filtered by `filterMovesForCheck()` — no move can get Black out of check.
- 4 Returns `MoveResult` with `isCheck=true, isCheckmate=true`. `Board` shows game over.

5. Method Mapping: Board → GameService

Current Board Method	New Location	Changes
<code>handleSquareClick()</code>	Board (simplified) + <code>GameService.handleAction()</code>	Board version becomes 3 lines: call <code>handleAction</code> , read result, update UI. All logic moves to <code>GameService</code> .
<code>isMovePossible()</code>	<code>GameService</code> (private)	Moved as-is. Only called internally by <code>handleAction</code> .
<code>moveSelectedPiece()</code>	<code>GameService.executeMove()</code> (private)	Renamed. Now also handles castling and en passant branching.
<code>capturePiece()</code>	Merged into <code>GameService.executeMove()</code>	Was already just calling <code>moveSelectedPiece</code> . No longer needed as separate method.
<code>selectPiece()</code>	Inside <code>GameService.handleAction()</code>	Selection logic is part of <code>handleAction</code> 's flow.
<code>getSelected()</code>	<code>GameService.getSelected()</code>	Moved. Board can call it if needed for highlighting.

6. New Methods Summary

Method	Purpose	Used By
<code>handleAction()</code>	Single entry point for all click actions	<code>Board.handleSquareClick()</code>
<code>getLegalMoves()</code>	Raw moves filtered for check safety (handles pins implicitly)	<code>handleAction()</code> , <code>hasNoLegalMoves()</code>

<code>filterMovesForCheck()</code>	Simulates each move, removes those leaving king in check	<code>getLegalMoves()</code>
<code>simulateMove()</code>	Creates board copy with move applied (non-destructive test)	<code>filterMovesForCheck()</code>
<code>isInCheck()</code>	Is the given color's king under attack?	<code>handleAction()</code> , <code>filterMovesForCheck()</code>
<code>isSquareAttacked()</code>	Can any piece of given color reach this square?	<code>isInCheck()</code> , <code>canCastle*()</code>
<code>findKing()</code>	Locates king position on board	<code>isInCheck()</code>
<code>isCheckmate()</code>	In check + no legal moves = game over	<code>handleAction()</code>
<code>isStalemate()</code>	Not in check + no legal moves = draw	<code>isDraw()</code>
<code>isDraw()</code>	Checks all draw conditions	<code>handleAction()</code>
<code>hasInsufficientMaterial()</code>	K vs K, K+B vs K, K+N vs K	<code>isDraw()</code>
<code>canCastleKingside()</code>	All castling conditions for kingside	<code>getLegalMoves()</code>
<code>canCastleQueenside()</code>	All castling conditions for queenside	<code>getLegalMoves()</code>
<code>isEnPassantMove()</code>	Is this move an en passant capture?	<code>getLegalMoves()</code> , <code>executeMove()</code>
<code>getEnPassantTarget()</code>	Returns capturable en passant square or null	<code>isEnPassantMove()</code> , <code>getLegalMoves()</code>

<code>switchTurn()</code>	WHITE ↔ BLACK	<code>handleAction()</code>
<code>isCorrectTurn()</code>	Enforces turn order	<code>handleAction()</code>
<code>updateCastlingRights()</code>	Marks king/rook as moved	<code>handleAction()</code>
<code>updateMoveHistory()</code>	Tracks position count + half-move clock	<code>handleAction()</code>
<code>boardToString()</code>	Serializes board for repetition detection	<code>updateMoveHistory()</code>

7. Board.handleSquareClick() - Before & After

Before (current - all logic in Board)

```
public void handleSquareClick(IndexPosition nextMove) {
    Piece targetPiece = piecesOnTheBoard[nextMove.getRow()][nextMove.getCol]

    if (selected == null) {
        if (targetPiece != null) {
            selected = targetPiece;
        }
        return;
    }

    if (targetPiece != null) {
        if (!selected.isEnemy(targetPiece)) {
            selected = targetPiece;
        } else if (isMovePossible(selected.getPossibleMoves(piecesOnTheBoard), nextMove)) {
            capturePiece(nextMove);
        }
    }
    else if (isMovePossible(selected.getPossibleMoves(piecesOnTheBoard), nextMove)) {
        moveSelectedPiece(nextMove);
    }
}
```

After (delegates to GameService)

```
public void handleSquareClick(IndexPosition nextMove) {
    MoveResult result = gameService.handleAction(nextMove, piecesOnTheBoard);

    refreshBoard();

    if (result.isCheckmate()) {
        // Show game over dialog
        JOptionPane.showMessageDialog(null,
            gameService.getCurrentTurn() + " is checkmated! Game over.");
    } else if (result.isDraw()) {
        JOptionPane.showMessageDialog(null,
            "Game is a draw: " + result.getDrawReason());
    } else if (result.isPromotion()) {
        // Show promotion picker, then call:
        // gameService.promotePawn(result.getPromotionPosition(), chosenPiece);
    }
}
```

```

        // refreshBoard();
    }
}

```

8. How Pins Work (No Explicit Pin Logic Needed)

A common misconception is that pins need dedicated detection code. With the `filterMovesForCheck()` approach, pins are handled **automatically**:

```

getLegalMoves(piece, board):
    rawMoves = piece.getPossibleMoves(board)    // piece doesn't know about check
    legalMoves = []

    for each move in rawMoves:
        simulatedBoard = simulateMove(piece, move, board) // copy board, apply move
        if NOT isInCheck(piece.color, simulatedBoard):    // does this expose king?
            legalMoves.add(move)                          // no -> legal move
                                                         // yes -> filtered out

    return legalMoves

```

If a bishop is pinned to its king by an enemy rook, any bishop move that isn't along the pin line will leave the king exposed. `simulateMove + isInCheck` catches this automatically. No separate "isPinned" flag or pin detection is required.

Note: The `simulateMove()` approach is simple and correct. For each candidate move, it creates a shallow copy of the 8x8 array, applies the move, and checks if the king is attacked. With at most ~27 possible moves per piece and 32 pieces to scan for attacks, this is fast enough for a chess game (no performance concern).

9. Implementation Priority Order

#	Feature	Dependencies	Why This Order
1	GameService skeleton + <code>handleAction</code> + <code>executeMove</code>	None	Establishes the architecture. Board delegates to GameService. Existing functionality preserved.
2	Turn enforcement (<code>currentTurn</code> , <code>switchTurn</code> ,	#1	Simple state, immediately testable, needed by everything else.

	isCorrectTurn)		
3	isSquareAttacked + findKing + isInCheck	#1	Foundation for all check-related logic.
4	simulateMove + filterMovesForCheck + getLegalMoves	#3	Once this works, pins are automatically handled.
5	Checkmate + Stalemate detection	#4	Uses getLegalMoves to check if any moves exist.
6	Castling	#3	Needs isSquareAttacked + castling state flags.
7	En passant	#4	Needs move history tracking + legal move filtering.
8	Draw conditions (insufficient material, repetition, 50-move)	#5	Final polish. Needs move history infrastructure.
9	Pawn promotion	#4	Needs UI interaction (promotion picker). Can be added last.